# Aim

To compute the **string similarity** for a given string **s**, defined as the sum of the lengths of the longest common prefixes between **s** and each of its suffixes (including itself).

# Algorithm

The solution uses the **Z-algorithm**, which efficiently computes how much of the string matches with its prefix starting from each position

1. **Input:** A string `s` of length `n`.
2. **Initialize:**
   - Create an array `Z` of size `n`.
   - `Z[i]` will store the length of the longest substring starting at `i` that matches the prefix of `s`.
   - Maintain a window `[left, right]` representing the current matching segment.
3. **Build Z-Array:**
   For each index `i` from `1` to `n-1`:
   - If `i` is inside the current window (`i ≤ right`):
     - Copy previously computed values:
     - `Z[i] = min(right - i + 1, Z[i - left])`
   - Extend the match manually:
     - Compare characters from the prefix with characters starting at `i`.
     - Increase `Z[i]` while characters match.
   - If the new match goes beyond `right`:
     - Update `left = i`
     - Update `right = i + Z[i] - 1`
4. **Compute Similarity Sum:**
   - The full string matches itself → contributes `n`
   - Add all values in the Z-array
5. **Return the total sum**

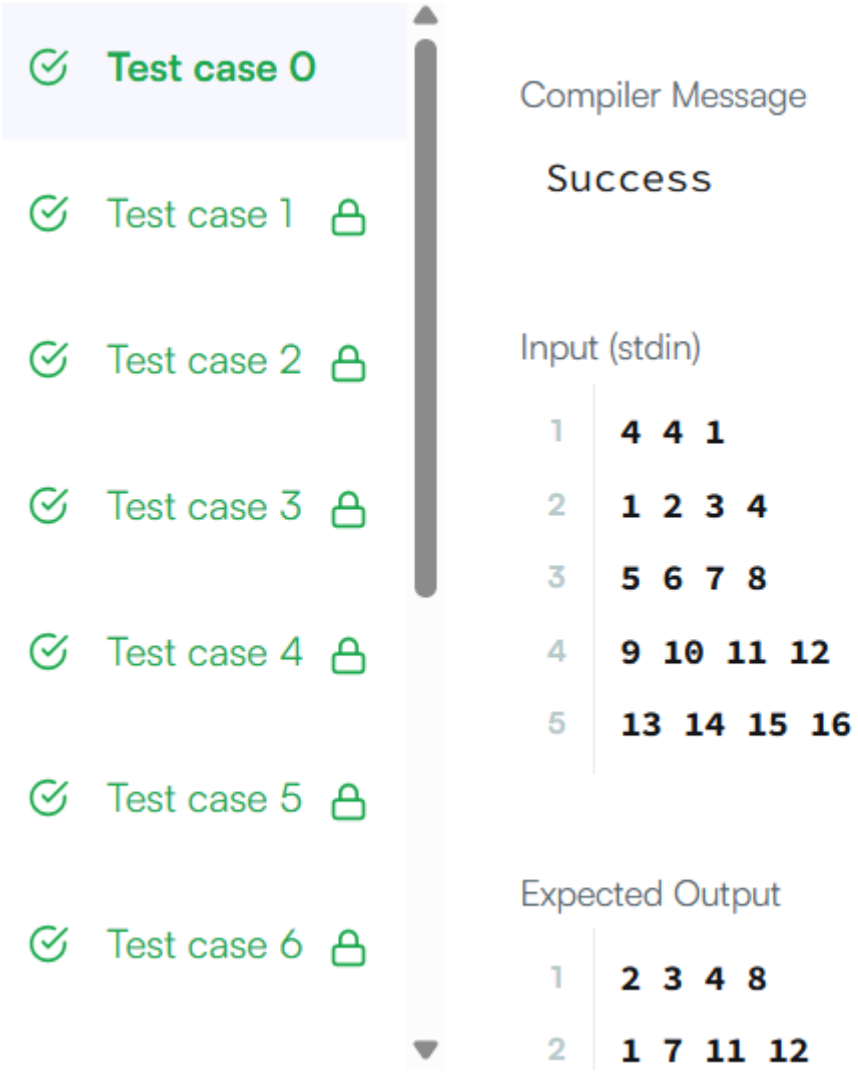
## Program:

```
import java.util.*;
import java.util.stream.*;

class Result {

    /*
     * Complete the 'stringSimilarity' function below.
     *
     * The function is expected to return a LONG.
     * The function accepts STRING s as parameter.
     */

    public static long stringSimilarity(String s) {

        int n = s.length();
        int[] Z = new int[n];
        int left = 0, right = 0;

        // Build Z-array
        for (int i = 1; i < n; i++) {

            if (i <= right) {
                Z[i] = Math.min(right - i + 1, Z[i - left]);
            }

            while (i + Z[i] < n && s.charAt(Z[i]) == s.charAt(i + Z[i])) {
                Z[i]++;
```

```
            }

            if (i + Z[i] - 1 > right) {
                left = i;
                right = i + Z[i] - 1;
            }
        }

        long sum = n; // include similarity with itself
        for (int val : Z) {
            sum += val;
        }

        return sum;
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter;

        String outputPath = System.getenv("OUTPUT_PATH");
        if (outputPath != null) {
            bufferedWriter = new BufferedWriter(new FileWriter(outputPath));
        } else {
```

```
        bufferedWriter = new BufferedWriter(new OutputStreamWriter(System.out));
    }

    int t = Integer.parseInt(bufferedReader.readLine().trim());

    IntStream.range(0, t).forEach(tItr -> {
        try {
            String s = bufferedReader.readLine();

            long result = Result.stringSimilarity(s);

            bufferedWriter.write(String.valueOf(result));
            bufferedWriter.newLine();

        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    });

    bufferedReader.close();
    bufferedWriter.close();
  }
}
```

**Output:**




**Result:** The function returns a **long value** representing the total similarity score.

## Aim

To find all strings in the array that are **substrings of another string** in the same array.

## Algorithm

Given an array `words`, identify every word that appears inside any other word.

**Steps:**

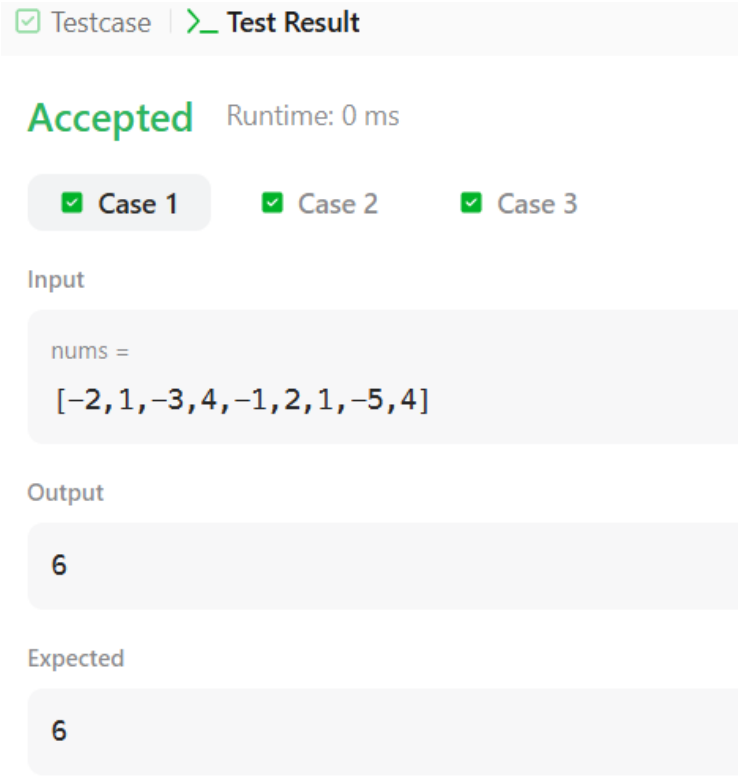
1. Let `n` be the number of words.
2. Create an empty list `ans` to store results.
3. For each word `words[i]`:
   - Compare it with every other word `words[j]` where `i ≠ j`.
   - Check if `words[j]` contains `words[i]` using `.contains()`.
4. If yes:
   - Add `words[i]` to the result list.
   - Stop checking further for that word (avoid duplicates).
5. Return the list `ans`.

## Program:

```
import java.util.*;

class Solution {

    public List<String> stringMatching(String[] words) {

        int n = words.length;

        List<String> ans = new ArrayList<>();



        for (int i = 0; i < n; ++i) {

            for (int j = 0; j < n; ++j) {

                if (i != j && words[j].contains(words[i])) {
```

```
                ans.add(words[i]);
                break;
            }
        }
    }

    return ans;
}
```

**Output:**




};

# Result

The function returns a **list of strings** that are substrings of another word in the array.

# Aim

To determine whether two strings share at least one common character (i.e., whether they have a non-empty common substring of length 1).

# Algorithm

The solution uses a HashSet to efficiently check character overlap.

### Steps:

1. **Input:** Two strings `s1` and `s2`.
2. **Store characters of first string**
   - Create a `HashSet<Character>`.
   - Add every character from `s1` into the set.
3. **Check characters of second string**
   - For each character in `s2`:
     - If it exists in the set → common character found.
4. **Return result**
   - If any common character exists → return `"YES"`
   - Otherwise → return `"NO"`

### Program:

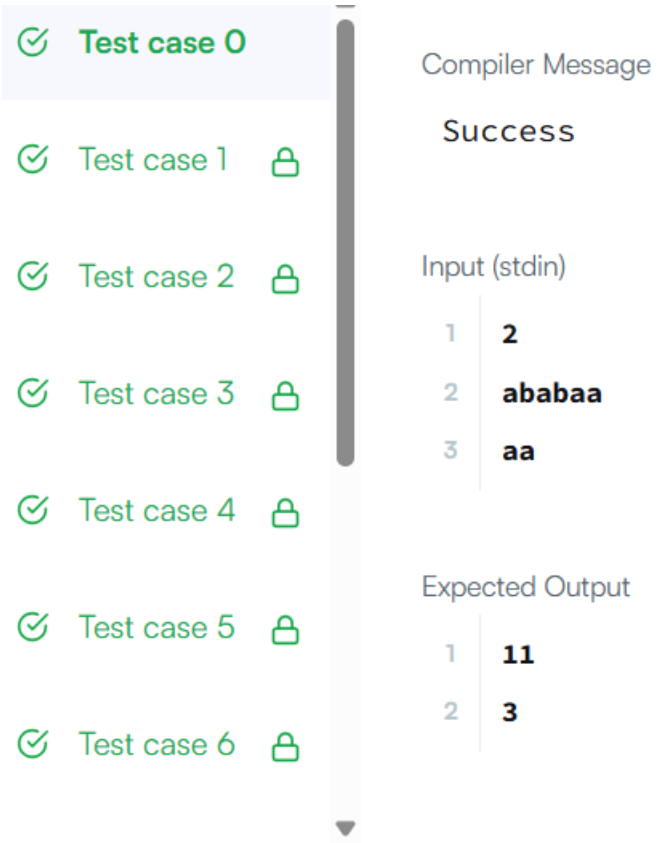
```
import java.io.*;

import java.util.*;

import java.util.stream.*;

class Result {
```

```
/*
 * Complete the 'twoStrings' function below.
 *
 * The function is expected to return a STRING.
 * The function accepts following parameters:
 *  1. STRING s1
 *  2. STRING s2
 */

public static String twoStrings(String s1, String s2) {

    HashSet<Character> set = new HashSet<>();

    // Add all characters from s1
    for (char c : s1.toCharArray()) {
        set.add(c);
    }

    // Check if any character of s2 is in s1
    for (char c : s2.toCharArray()) {
        if (set.contains(c)) {
            return "YES";
        }
    }

    return "NO";
```

```
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter;

        String outputPath = System.getenv("OUTPUT_PATH");
        if (outputPath != null) {
            bufferedWriter = new BufferedWriter(new FileWriter(outputPath));
        } else {
            bufferedWriter = new BufferedWriter(new OutputStreamWriter(System.out));
        }

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qItr -> {
            try {
                String s1 = bufferedReader.readLine();
                String s2 = bufferedReader.readLine();

                String result = Result.twoStrings(s1, s2);

                bufferedWriter.write(result);
                bufferedWriter.newLine();
```

```
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

**Output:**




# Result

The function returns:

- `"YES"` → if the two strings share at least one character

- `"NO"` → if they share none

# Aim

To check whether one string (`goal`) can be obtained by rotating another string (`s`).

# Algorithm

This solution uses a clever string property of rotations.

### Steps:

1. **Check lengths**
   - If `s` and `goal` have different lengths → rotation is impossible → return `false`.
2. **Concatenate the string**
   - Create a new string: `s + s`
   - This contains all possible rotations of `s`.
3. **Check substring**
   - If `goal` appears inside `s + s`, then `goal` is a rotation of `s`.
4. **Return result**
   - If found → `true`
   - Otherwise → `false`

## Program:

```
class Solution {
    public boolean rotateString(String s, String goal) {
        if (s.length() != goal.length()) {
            return false;
        }
        return (s + s).contains(goal);
    }
}
```

**Output:**

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =

"abab"

Output

true

Expected

true

# Result

Thus,The function returns and `true` → if `goal` is a rotation of `and false` → otherwise verified

# Aim

To count how many characters were altered in a received message that should consist of repeated **"SOS"** signals.

# Algorithm

1. The original message is a repetition of `"SOS"` (e.g., SOS, SOSSOS, SOSSOSSOS…).
2. Traverse the string character by character.
3. For each index `i`:
   - If `i % 3 == 0` → expected `'S'`
   - If `i % 3 == 1` → expected `'O'`
   - If `i % 3 == 2` → expected `'S'`
4. Compare the actual character with the expected one.
5. If different → increment the error counter.
6. Return the total count of altered characters.

# Program:

```
import java.io.*;
import java.util.*;

class Result {

    /*
     * Complete the 'marsExploration' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts STRING s as parameter.
     */

    public static int marsExploration(String s) {

        int count = 0;

        for (int i = 0; i < s.length(); i++) {

            if (i % 3 == 0) {          // Expected 'S'
                if (s.charAt(i) != 'S') count++;
            }
```

```
            else if (i % 3 == 1) {     // Expected 'O'
                if (s.charAt(i) != 'O') count++;
            }
            else {                     // Expected 'S'
                if (s.charAt(i) != 'S') count++;
            }
        }

        return count;
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter;

        String outputPath = System.getenv("OUTPUT_PATH");
        if (outputPath != null) {
            bufferedWriter = new BufferedWriter(new FileWriter(outputPath));
```

```
        } else {
            bufferedWriter = new BufferedWriter(new OutputStreamWriter(System.out));
        }

        String s = bufferedReader.readLine();

        int result = Result.marsExploration(s);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

**Output:**

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Test case 7

Compiler Message

```
Success
```

Input (stdin)

```
2
hello
world
hi
world
```

Expected Output

```
YES
NO
```

# Result

Returns an **integer** representing the number of changed characters.

# Aim

To find all starting indices in string **s** where an anagram of string **p** occurs.

# Algorithm

1. Create two frequency arrays:
   - `pFreq` → frequency of characters in `p`
   - `sFreq` → frequency of current window in `s`
2. Fill `pFreq` using characters of `p`.
3. Use a sliding window on `s`:
   - Expand window by moving `right`
   - Update `sFreq` for the new character
4. Maintain window size equal to `p.length()`:
   - If window becomes larger → remove left character and move `left`
5. Compare `sFreq` and `pFreq`:
   - If equal → current window is an anagram → add `left` index to result
6. Continue until end of string

# Program:

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {

        List<Integer> ans = new ArrayList<>();

        int[] pFreq = new int[256];
        int[] sFreq = new int[256];

        int left = 0, len;
        for (int i = 0; i < p.length(); i++) {
            pFreq[p.charAt(i) - 'a']++;
        }
        for (int right = 0; right < s.length(); right++) {
            sFreq[s.charAt(right) - 'a']++;

            len = right - left + 1;
            while (len > p.length()) {
                sFreq[s.charAt(left) - 'a']--;
                left++;
                len = right - left + 1;
            }
```

```
            if (isAnagrams(sFreq, pFreq)) {
                ans.add(left);
            }
        }

        return ans;
    }

    private boolean isAnagrams(int[] sFreq, int[] pFreq) {
        for (int i = 0; i < 256; i++) {
            if (sFreq[i] != pFreq[i]) return false;
        }
        return true;
    }
}
```

**Output:**

Accepted Runtime: 0 ms

Case 1 Case 2

Input

s =

```
"abcde"
```

goal =

```
"cdeab"
```

Output

```
true
```

Expected

```
true
```

**Result**:

Thus, To find all starting indices in string **s** where an anagram of string **p** occurs.

# Aim

To find the index of a character which, when removed, makes the string a palindrome.
If the string is already a palindrome or cannot be made one by removing a single character, return **-1**.

# Algorithm

1. Use two pointers:
   - `left` at start
   - `right` at end
2. Move inward while characters match.
3. On first mismatch:
   - Check if removing the left character makes the remaining substring a palindrome
     → If yes, return `left`
   - Otherwise check removing the right character
     → If yes, return `right`
4. If neither works → return `-1`
5. If loop finishes without mismatch → string already palindrome → return `-1`

## Program:

```
import java.io.*;

import java.util.*;

import java.util.stream.*;

class Result {



    /*

     * Complete the 'palindromeIndex' function below.

     *

     * The function is expected to return an INTEGER.

     * The function accepts STRING s as parameter.
```

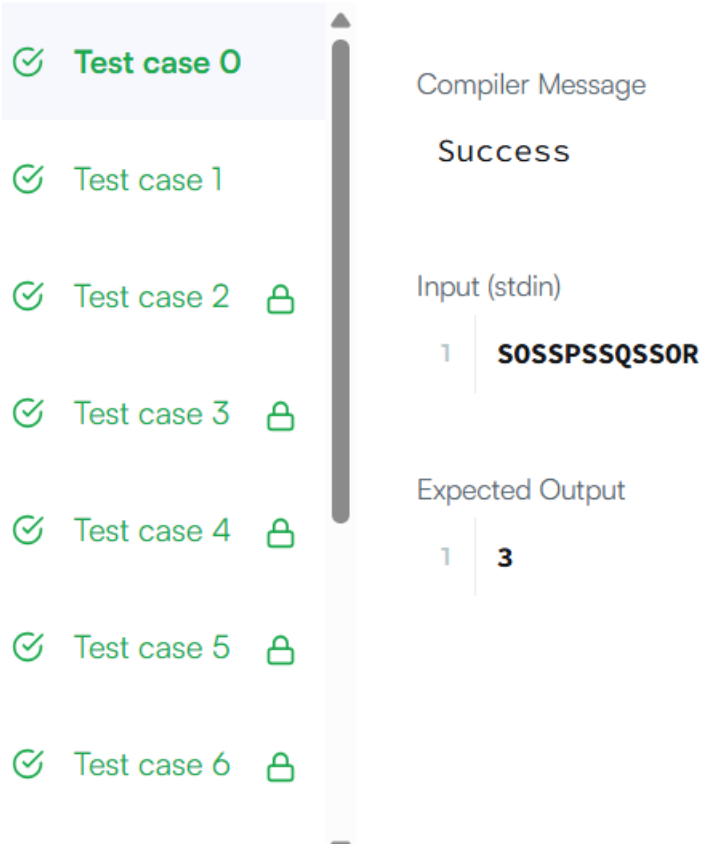
```
 */

public static int palindromeIndex(String s) {

    int left = 0;
    int right = s.length() - 1;

    while (left < right) {

        if (s.charAt(left) == s.charAt(right)) {
            left++;
            right--;
        } else {

            // Remove left character
            if (isPalindrome(s, left + 1, right)) {
                return left;
            }

            // Remove right character
            if (isPalindrome(s, left, right - 1)) {
```

```
                return right;
            }

            return -1;
        }
    }

    return -1; // Already palindrome
  }

  private static boolean isPalindrome(String s, int l, int r) {
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++;
        r--;
    }
    return true;
  }
}

public class Solution {
```

```
    public static void main(String[] args) throws IOException {


        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter;


        String outputPath = System.getenv("OUTPUT_PATH");

        if (outputPath != null) {

            bufferedWriter = new BufferedWriter(new FileWriter(outputPath));

        } else {

            bufferedWriter = new BufferedWriter(new OutputStreamWriter(System.out));

        }


        int q = Integer.parseInt(bufferedReader.readLine().trim());


        IntStream.range(0, q).forEach(qItr -> {

            try {

                String s = bufferedReader.readLine();


                int result = Result.palindromeIndex(s);


                bufferedWriter.write(String.valueOf(result));
```

```
            bufferedWriter.newLine();


        } catch (IOException ex) {

            throw new RuntimeException(ex);

        }

    });


    bufferedReader.close();

    bufferedWriter.close();

  }

}
```

**Output:**




**Result:**

This program finds an index whose removal makes the string a palindrome, otherwise returns minus one.

# Aim

To find the first index where string needle appears in haystack.

# Algorithm

1. Slide a window of size `needle.length()` over `haystack`.
2. For each position, extract the substring.
3. Compare it with `needle`.
4. If equal → return the starting index.
5. If no match found → return `-1`.

## Program:

```
class Solution {
    public int strStr(String haystack, String needle) {
        for(int i = 0, j = needle.length(); j<=haystack.length(); i++,j++){
            if(haystack.substring(i,j).equals(needle)){
                return i;
            }
        }
        return -1;
    }
}
```

**Accepted** Runtime: 0 ms

Case 1 Case 2

Input

s =

```
"cbaebabacd"
```

p =

```
"abc"
```

Output

```
[0,6]
```

Expected

```
[0,6]
```

Result :

Thus, find the first index where string needle appears in haystack.

# Aim

To compute the **length of the longest palindromic substring** for each rotation of a given string `s`.

- A **rotation** moves the first character to the end.
- Output a list of integers representing the maximum palindrome length for each rotation.

# Algorithm

1. **Input:** string `s` of length `n`.
2. **Create a doubled string:** `t = s + s`. This allows easy generation of all rotations via substrings of length `n`.
3. **For each rotation starting at index `i = 0 to n-1`:**
    - Consider substring `rotation = t[i : i + n]`.
    - Find the **longest palindromic substring** within this substring:
        - Use **center expansion**: for each character, expand around it for odd and even length palindromes.
        - Keep track of maximum palindrome length.
    - Add maximum length to the result list.
4. **Return** the list of lengths for all rotations.

**Optimization notes:**

- Using the doubled string avoids repeated substring allocations.
- Expanding from the center for palindromes is **$O(n^2)$** per rotation.
- Full Manacher's algorithm can reduce per-rotation computation to **O(n)** in advanced solutions

## Program:

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;

class Result {

    public static List<Integer> circularPalindromes(String s) {

        int n = s.length();
        String t = s + s;
        List<Integer> res = new ArrayList<>(n);

        for (int start = 0; start < n; start++) {

            int max = 1;
            int L = start;
            int R = start + n - 1;

            for (int center = L; center <= R; center++) {

                // Odd length
                int l = center, r = center;
```

```
            while (l >= L && r <= R && t.charAt(l) == t.charAt(r)) {
                max = Math.max(max, r - l + 1);
                l--;
                r++;
            }

            // Even length
            l = center;
            r = center + 1;
            while (l >= L && r <= R && t.charAt(l) == t.charAt(r)) {
                max = Math.max(max, r - l + 1);
                l--;
                r++;
            }
        }

        res.add(max);
    }

    return res;
  }
}
```

```
public class Solution {
    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(
                new FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());
        String s = bufferedReader.readLine();

        List<Integer> result = Result.circularPalindromes(s);

        bufferedWriter.write(
                result.stream()
                        .map(Object::toString)
                        .collect(joining("\n")) + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

**Output:**

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

```
Success
```

Input (stdin)

```
3
aaab
baa
aaa
```

Expected Output

```
3
0
-1
```

Result: For each rotation of the string, return the length of its longest palindromic substring